EDS Lab Assignments

Practical 2

Name: Yogeshwari Thombare

PRN:202501040053

2.1.1 List Operations

Problem Statement:

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit**: Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

```
l1 = []
while True:
        print("1. Add")
        print("2. Remove")
        print("3. Display")
        print("4. Quit")

        n=int(input("Enter choice: "))
        if n==1:
                add = int(input("Integer: "))
                l1.append(add)
```

```
        print(f"List after adding: {l1}")
elif n==2:
        if len(l1)==0:
                print("List is empty")
        elif len(l1)!=0:
                remove=int(input("Integer: "))
                if remove not in l1:
                        print("Element not found")
                else:
                        l1.remove(remove)
                        print(f"List after removing: {l1}")
elif n==3:
        if len(l1)==0:
                print("List is empty")
        else:
                print(l1)
elif n==4:
        break
else:
        print("Invalid choice")
```

Output:

Average time	Maximum time
0.012 s	**0.017 s**
12.00 ms	17.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 17 ms Debug

Expected output	Actual output
Enter the first value: 1.0	Enter the first value: 1.0
Enter the second value: 2.3	Enter the second value: 2.3
Enter the third value: hello	Enter the third value: hello
List1: ['1.0', '2.3', 'hello']	List1: ['1.0', '2.3', 'hello']
Enter the first value: hi	Enter the first value: hi
Enter the second value: 8.3	Enter the second value: 8.3
Enter the third value: 9.6	Enter the third value: 9.6
List2: ['hi', '8.3', '9.6']	List2: ['hi', '8.3', '9.6']
List after Concatenation: ['1.0', '2.3', 'hello', 'hi', '8.3', '9.6']	List after Concatenation: ['1.0', '2.3', 'hello', 'hi', '8.3', '9.6']

Average time	Maximum time
0.048 s	**0.069 s**
47.67 ms	69.00 ms

✔ 2 out of 2 shown test case(s) passed

✔ 1 out of 1 hidden test case(s) passed

```
Integer: 26
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 11
List after removing: [25]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 25
List after removing: []
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
List is empty
```

```
Integer: 26
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 11
List after removing: [25]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 25
List after removing: []
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
List is empty
```

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 8
Invalid choice
1. Add
2. Remove
3. Display
4. Quit
```

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 8
Invalid choice
1. Add
2. Remove
3. Display
4. Quit
```

2.1.2 Dictionary Operations

Problem Statement:
Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Code:

```
# Initial dictionary with 10 predefined records
student = {
    1: "Amit",
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}

# write your code here..
print("Original Dictionary:", student)

new_key = int(input())
new_value = input()
student[new_key] = new_value

print("After Insertion:", student)

update_key = int(input())
new_val = input()
if update_key in student:
        student[update_key] = new_val
print("After Update:", student)

del_key = int(input())
if del_key in student:
        del student[del_key]
print("After Deletion:", student)

print("Traversing Dictionary:")
for key, value in student.items():
        print(f"{key} : {value}")
```

Output:

Average time	Maximum time
0.017 s	0.018 s
16.50 ms	18.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 18 ms Debug

Expected output	Actual output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}	Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11	11
Suresh	Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}	After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
3	3
Karthik	Karthik
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}	After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
5	5
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}	After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
Traversing Dictionary:	Traversing Dictionary:
1 : Amit	1 : Amit

Traversing Dictionary:	Traversing Dictionary:
1 : Amit	1 : Amit
2 : Riya	2 : Riya
3 : Karthik	3 : Karthik
4 : Neha	4 : Neha
6 : Pooja	6 : Pooja
7 : Rahul	7 : Rahul
8 : Sneha	8 : Sneha
9 : Vikram	9 : Vikram
10 : Anjali	10 : Anjali
11 : Suresh	11 : Suresh

2.2.1 Linear Search Technique

Problem Statement:
Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

Code:

```
def linear_search(arr, key):
	for i in range(len(arr)):
		if arr[i] == key:
			return i
	return -1

arr = list(map(int,input().split()))
key = int(input())

result = linear_search(arr, key)

if result != -1:
	print(result)
else:
	print("Not found")
```

Output:

Average time
0.009 s
9.00 ms

Maximum time
0.009 s
9.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 9 ms Debug

Expected output	Actual output
1 2 3 4 3 5 6	1 2 3 4 3 5 6
3	3
2	2

Test case 2 9 ms Debug

Expected output	Actual output
1 2 3 4 5 6 7 8 9 19	1 2 3 4 5 6 7 8 9 19
20	20
Not found	Not found

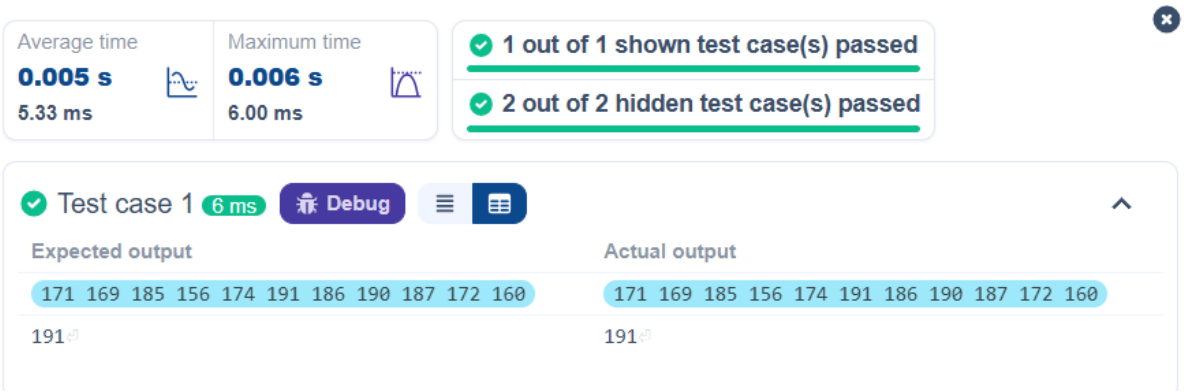
2.2.2 Captain of the Team

Problem Statement:
You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format
The output should be the height (in centimeters) of the tallest player.

Code:

```
heights = list(map(int,input().split()))
tallest_player = max(heights)
print(tallest_player)
```

Output: